

Параллельное программирование

Домашняя работа №4. Сравнение исполнителей задач

Выполнил: Деревянкин Тимофей
Группа: 24-по-1
Дата: 08.10.2026

Цель и реализация

Реализовать обёртку FixedThreadPool, исполняющую ShutdownableExecutor, сравнить её с ThreadPerTask, RoundRobin и WorkStealing. Повторить замеры после замены sleep на вычислительную функцию blackHole(difficulty).

Обёртка делегирует execute в Executors.newFixedThreadPool; shutdown закрывает приём и ждёт awaitTermination. RoundRobin распределяет задачи по отдельным очередям. WorkStealing при пустой своей очереди берёт задачу с конца чужой. Счётчик pending учитывает ожидающие и выполняемые задачи, чтобы shutdown не завершал работников раньше времени. ThreadPerTask создаёт отдельный платформенный поток.

Условия эксперимента

Windows 11, Intel i5-11400F (6 физических / 12 логических ядер), Temurin 21.0.12.1. 240 задач, 6 работников в каждом пуле, средняя сложность 4, сумма 960, seed=42. Для каждого случая: 1 прогрев и 3 измерения, результат - медиана. Вместо 100000 задач выбрано 240, чтобы можно было запустить ThreadPerTask. Порядок исполнителей перемешивается, массив сложности для них одинаков.

UNIFORM: равномерное распределение. PERIODIC: каждая шестая задача тяжёлая, остальные нулевые. PARETO: редкие длинные задачи, параметр формы 1.5, ограничение выбросов. Все массивы нормированы до одинаковой суммы. В SLEEP единица - миллисекунда; в CPU единица - 200000 шагов.

Медианы полного времени, мс

Режим / распределение	ThreadPerTask	RoundRobin	WorkStealing	FixedPool
SLEEP / UNIFORM	27.898	235.460	206.310	196.179
SLEEP / PERIODIC	41.538	991.292	178.648	174.741
SLEEP / PARETO	288.391	428.434	412.321	398.057
CPU / UNIFORM	36.893	74.598	63.672	54.553
CPU / PERIODIC	46.688	324.747	64.681	56.391
CPU / PARETO	114.519	133.078	133.137	120.448

Полное время включает создание исполнителя, подачу задач и ожидание завершения. Подготовка задач, печать и запись CSV исключены. Все 72 измеряемых запусков завершили ровно по 240 задач без зарегистрированных ошибок.

Линейная вычислительная нагрузка

blackHole(difficulty) выполняет точно $200000 \times \text{difficulty}$ итераций с зависимыми целочисленными операциями сдвига, умножения и сложения. Сложность $O(\text{difficulty})$. Результат записывается в volatile-поле один раз после цикла: это препятствует удалению вычисления JIT. Переполнение long внутри смешивания намеренное; отрицательная сложность запрещена, переполнение счётчика итераций проверяется multiplyExact.

Медианы времени подачи, мс

Режим / распределение	ThreadPerTask	RoundRobin	WorkStealing	FixedPool
SLEEP / UNIFORM	12.574	0.342	0.316	0.669
SLEEP / PERIODIC	11.947	0.682	0.384	0.409
SLEEP / PARETO	9.734	1.029	0.283	0.512
CPU / UNIFORM	8.218	0.301	0.311	0.449
CPU / PERIODIC	8.776	0.228	0.259	0.362
CPU / PARETO	8.673	0.379	0.278	0.372

Время подачи также включает конструктор исполнителя. Задачи могут выполняться параллельно с подачей, поэтому это не отдельная непересекающаяся стадия.

Результаты и выводы

При PERIODIC тяжёлые задачи RoundRobin оказываются в одной очереди. FixedPool быстрее него примерно в 5.7 раза для sleep и в 5.8 раза для CPU. Общая очередь и кража задач позволяют занять простаивающих работников. FixedPool быстрее двух самодельных пулов во всех шести случаях данной серии.

ThreadPerTask оказался быстрее на этой небольшой выборке: ожидания перекрываются, а вычисления могут занять больше шести потоков и использовать логические ядра. Количество работающих потоков не выровнено с пулами. Это не доказывает эффективность создания 100000 потоков. При PARETO отдельная длинная задача остаётся неделимой и может определять время завершения независимо от очередей.

Абсолютные времена SLEEP и CPU соответствуют разным типам работы: сложность CPU не задаёт длительность в миллисекундах. Это учебный стенд, не JMH; три повтора не исключают влияние JIT, планировщика, разрешения таймеров Windows и фоновых программ.

Проверки и запуск

Самопроверка контролирует выполнение ровно один раз, отказ после закрытия, пустой и повторный shutdown, конкурирующие execute/shutdown, сохранение прерывания и нормировку распределений. Проверки прошли для всех четырёх исполнителей.

```
java -jar WorkStealing.jar --check
java -jar WorkStealing.jar --csv benchmark.csv
```

Документация: Oracle Java SE 21, ExecutorService и Executors.
docs.oracle.com: ExecutorService; Executors.